DP-OPS-0010: Architectural Specification for Root Living Surfaces and Operational Governance Systems

1. Executive Summary and Systemic Philosophy

The Stela repository, as defined by its core constitution in TRUTH.md, operates on a rigorous "explainability-first" doctrine that prioritizes structure over memory and governance over heroics.[1] The current operational state relies heavily on a single "Living Surface"—TASK.md—to manage the active work thread, dispatch packets (DPs), and continuity logs.[1] While this single-threaded model successfully manages immediate execution intent, the system currently lacks equivalent high-level mechanisms for managing persistent agent capabilities ("Skills") and long-duration workstreams ("Projects").

This report serves as the exhaustive architectural specification for **DP-OPS-0010: Establish Root Living Surfaces**. It details the necessary transformation of the repository from a task-runner environment into a multi-dimensional operating system capable of tracking *what is being done* (Task), *what capabilities are available* (Skill), and *what long-term initiatives are active* (Project). This evolution requires the elevation of SKILL.md and PROJECT.md to the status of "Root Living Surfaces," modeled strictly after the governance patterns and freshness gates established in TASK.md.

Furthermore, to support this architectural expansion, this specification defines the implementation of the S-LEARN-01 acquisition tool, the establishment of a formal project registry (ops/lib/project/REGISTRY.md), and the standardization of decentralized project definitions via stela.md. These interventions are designed to be strictly language-agnostic, adhering to the "No PHP references" constraint to ensure the governance layer remains distinct from the legacy runtime it manages.

1.1 The "Living Surface" Paradigm

In the Stela architecture, a "Living Surface" is distinct from static documentation or historical logs. Static documentation (docs/) explains the system, and logs (SoP.md) record the history of state changes.[1] A Living Surface, however, acts as a mutable state container—a control plane that dictates current system behavior and focus.

The current architecture presents a governance imbalance:

- **Constitution (Immutable):** TRUTH.md defines the laws and invariants.[1]
- **Ledger (Append-Only):** SoP.md tracks the history of governance changes and what has shipped.[1]

- **Active Surface (Mutable):** TASK.md tracks the current objective, the active Dispatch Packet (DP), and the execution log for the current thread.[1]

This configuration leaves "Skills" (learned patterns and tools) and "Projects" (folders in projects/) as "dark matter"—they exist on the filesystem but lack a unified, addressable control plane in the root context. DP-OPS-0010 addresses this by introducing two new pillars to the root directory:

1. **SKILL.md (Capability Surface):** A living registry of verified agent capabilities, acting as the memory bank for the "Contractor" agent defined in AGENTS.md.[1]
2. **PROJECT.md (Initiative Surface):** A living dashboard for the projects/ directory, transforming it from a passive storage container into a governed portfolio of active workstreams.

1.2 Strategic Objectives and Anti-Drift Governance

The primary strategic driver for this architecture is "Anti-Drift Governance," a core operational directive defined in AGENTS.md. Drift occurs when the repository state diverges from the mental model of the operators or when logic is not aligned with TRUTH.md coordinates.[1] Without SKILL.md, an agent finding a solution in one session has no standardized place to record that "Skill" for future sessions, leading to capability drift (forgetting). Without PROJECT.md, the status of a project (Active, Archived, Maintenance) is implicit and relies on human memory, leading to governance drift.

By reifying these abstract concepts into concrete Markdown files at the repo root, the system enforces the "Context is Truth" doctrine found in docs/CONTEXT.md: "If it's not in the Context, it doesn't exist".[1]

2. Architectural Analysis of the Current State

2.1 The "Four-Line Heuristic" and Root Topology

TRUTH.md establishes the "Stela Filing Doctrine" via the 4-Line Heuristic [1]:

1. **Does it RUN the system?** -> ops/
2. **Does it EXPLAIN the system?** -> docs/
3. **Is it the WORK itself?** -> projects/
4. **Is it TEMPORARY trash?** -> storage/

The Root directory currently contains the "Constitution" (TRUTH.md), the "Ledger" (SoP.md), and the "Work Surface" (TASK.md). DP-OPS-0010 respects this heuristic but refines the definition of the "Root" layer. The Root layer acts as the kernel of the operating system. Adding SKILL.md and PROJECT.md asserts that *Capability* and *Initiative* are as fundamental to the system as *Constitution* and *Task*. This aligns with the AGENTS.md staffing protocol, which distinguishes between the "Operator" (Human), "Integrator" (Lead AI), and "Contractor"

(Guest AI).[1] TASK.md is the Contractor's daily sheet; SKILL.md is the Contractor's resume; PROJECT.md is the Integrator's roadmap.

2.2 Gap Analysis: The Missing "Memory"

The current dump-full-main manifest reveals a significant gap in knowledge retention.[1] While docs/library/MANUAL.md provides human-readable commands [1], there is no machine-readable registry of "verified patterns." If an agent successfully refactors a documentation set using a specific sequence of sed commands, that knowledge is currently buried in a closed TASK.md log or lost entirely.

The requirement to create S-LEARN-01 (a tool) and SKILL.md (a surface) directly addresses this. It transforms the system from one that *has* documentation to one that *possesses* skills.

2.3 Gap Analysis: The Ungoverned Project Space

The projects/README.md file contains a single line: "This directory holds project payloads".[1] This minimalism, while philosophically consistent with keeping the root clean, fails to support complex lifecycle management. There is no mechanism to:

- Enforce standard files (like README or LICENSE) within a project beyond the basic verify_tree.sh check.[1]
- Track the owner of a project.
- Link a specific DP in TASK.md to a specific project in projects/.

The introduction of PROJECT.md (Root Index) and stela.md (Local Manifest) creates a linked data structure. PROJECT.md serves as the queryable index, while stela.md serves as the decentralized node definition, ensuring scalable governance without a monolithic database.

3. Specification: SKILL.md (The Capability Surface)

3.1 Design Philosophy

SKILL.md serves as the Single Source of Truth (SSOT) for agent capabilities. It must be modeled after TASK.md because TASK.md has proven successful in maintaining "Freshness Gates" and "Context Loads".[1] A Skill is not just a text description; it is an executable logic block that requires specific context to run safely.

3.2 Structural Definition

The file structure will mirror the sectioning logic of TASK.md but adapted for capability retention rather than task execution.

Section	TASK.md Equivalent	Purpose in SKILL.md

Header	Active Context	Defines status, owner (Integrator), and last update timestamp.
0) Pre-Flight	Required Context	Lists the absolute requirements for using *any* skill (e.g., TRUTH.md).
1) Development	Active Context	Tracks if a skill is currently being refined or refactored.
2) Library	Current DP	The catalog of verified skills, organized by ID (e.g., S-OPS-001).
3) Log	Work Log	A chronological ledger of when skills were added, deprecated, or verified.

3.3 The "Skill Context" Requirement

In TASK.md, the worker must confirm they have loaded specific files (OPEN, TRUTH, etc.) before executing.[1] Similarly, SKILL.md will enforce a "Skill Context." For example, a skill named S-DOC-REFACTOR might strictly require docs/CONTEXT.md to be loaded to prevent it from hallucinating non-existent paths. This "Safety First" approach is derived from AGENTS.md logic standards regarding "Anti-Drift Governance".[1]

3.4 Integration with S-LEARN-01

The SKILL.md surface is the database for the S-LEARN-01 tool. This tool will automate the entry of new skills, preventing formatting errors that could break the file's machine-readability. (Detailed tool specification follows in Section 6).

4. Specification: PROJECT.md (The Initiative Surface)

4.1 Design Philosophy

PROJECT.md acts as the bridge between the ephemeral nature of TASK.md and the persistent nature of the projects/ directory. It is the "Air Traffic Control" tower for the repository.

4.2 Structural Definition

Like SKILL.md and TASK.md, this surface must be "Living." It must reflect the *current* state of the portfolio.

Section	Content	Governance Function
Header	Status, Owner, Date	Establishes accountability.
1) Registry	Counts (Active vs Archive)	Provides a high-level health metric.
2) Watchlist	List of Active Projects	Prioritizes attention; "Active" means currently consuming DPs.
3) Log	Lifecycle Events	records initialization, archival, and reactivation events.

4.3 The Registry Backing (ops/lib/project/REGISTRY.md)

While PROJECT.md is the dashboard, ops/lib/project/REGISTRY.md is the comprehensive database. This separation prevents the root file from becoming too large. The Registry will contain a table of *all* projects, their IDs, paths, and creation dates. This follows the pattern of docs/library/INDEX.md acting as a registry for documentation.[1]

5. Specification: stela.md (The Project Manifesto)

5.1 Decentralized Governance

The requirement to "Create stela.md in project roots" introduces a pattern of decentralized governance. Instead of a central file knowing everything about every project, each project describes itself. This is critical for scaling.

5.2 Content Schema (Language Agnostic)

To ensure "No PHP references" and broad applicability, the stela.md schema focuses on *metadata* and *intent* rather than implementation details.

Proposed Schema for stela.md:

- **Project ID:** (e.g., P-CORE-01)
- **Slug:** (e.g., platform-hardening)

- **Type:** (e.g., module, library, service, dataset) - generic types.
- **Status:** (e.g., active, archived)
- **Dependencies:** List of other Stela project IDs.
- **Context:** Specific files required to work on this project (extending the global CONTEXT.md).

This file essentially acts as a localized CONTEXT.md for that specific project, allowing ops/bin/open to eventually read it and smart-load necessary context.

6. Tooling Implementation: S-LEARN-01

6.1 Tool Identity and Mapping

The request asks to "Create tool S-LEARN-01". In the Stela nomenclature (evident in ops/bin/open, ops/bin/dump), binary tools are lowercase verbs. Therefore, S-LEARN-01 will be the *System ID* for the binary script located at ops/bin/s-learn.

6.2 Functional Specification

The ops/bin/s-learn script will be a Bash script (consistent with ops/bin/dump and open [1]) that facilitates the appending of new skills to SKILL.md.

Logic Flow:

1. **Input:** Accepts arguments for Skill ID, Title, Trigger, and Description.
2. **Validation:** Checks SKILL.md to ensure the ID is unique.
3. **Formatting:** Formats the input into the strict Markdown block required by SKILL.md (matching the TASK.md aesthetic).
4. **Appending:** Appends the new block to the "SKILL LIBRARY" section.
5. **Logging:** Adds a timestamped entry to the "SKILL LOG" section of SKILL.md.

Code Considerations:

- Must utilize set -euo pipefail for robustness.[1]
- Must be executable.
- Must not introduce any language dependencies other than Bash.

7. Library Expansion (docs/library/)

7.1 The Role of Libraries

The docs/library/ directory contains "curated operator guides".[1] The current INDEX.md lists MANUAL.md and MAP.md.[1] DP-OPS-0010 requires the creation of three new libraries:

1. **TASKS.md:** A manual explaining the lifecycle of a Dispatch Packet, how to fill out TASK.md, and the "Freshness Gate" mechanics.

2. **SKILLS.md:** A manual explaining how to use S-LEARN-01, how to query SKILL.md, and the criteria for a "Verified Skill."
3. **PROJECTS.md:** A manual explaining the Project Lifecycle (Intake -> Active -> Archive), the role of PROJECT.md, and how to write a stela.md manifest.

7.2 Registry Enforcement

These new files must be registered in docs/library/INDEX.md. The tools/lint_library.sh script enforces that "No Ghosts" exist (files on disk not in the index) and "No Dead Ends" exist (index entries not on disk).[1] Therefore, updating INDEX.md is a critical part of the execution plan to pass the repo-gates.

8. The "Context is Truth" Imperative

8.1 Analysis of docs/CONTEXT.md

docs/CONTEXT.md states: "Context is not memory; it is a file." and "If it's not in the Context, it doesn't exist.".[1] Currently, the "Core Set" in ops/lib/manifests/CONTEXT.md includes TRUTH.md, SoP.md, TASK.md, and AGENTS.md.[1]

8.2 Justification for Update

The user query states: "Update ops/lib/manifests/CONTEXT.md only if needed."

Analysis: If SKILL.md and PROJECT.md are "Root Living Surfaces" modeled after TASK.md, they are by definition part of the "Canonical Context."

- If SKILL.md is not in the context, agents cannot "remember" skills.
- If PROJECT.md is not in the context, agents cannot "see" the active roadmap.
  Conclusion: It is strictly needed. The update will add SKILL.md and PROJECT.md to the mandatory load list in the manifest. This ensures that every time ops/bin/open is run, the agent is rehydrated with the full knowledge of Skills and Projects.

9. Execution Plan (DP-OPS-0010 Specification)

The following section outlines the precise content of the Dispatch Packet (DP) to be drafted. This plan adheres to the TASK.md template structure.[1]

9.1 DP-OPS-0010: Establish Root Living Surfaces

Objective:

To transform the repository governance from a task-only model to a comprehensive Task-Skill-Project model by establishing root living surfaces, creating the S-LEARN-01 acquisition tool, and formalizing the project registry.

Scope & Allowlist:

The scope is strictly limited to the creation of the new governance surfaces and the supporting tooling.

- **Target Files (Allowlist):**
  - SKILL.md (new)
  - PROJECT.md (new)
  - ops/bin/s-learn (new)
  - ops/lib/project/REGISTRY.md (new)
  - projects/stela.md (new - template)
  - docs/library/TASKS.md (new)
  - docs/library/SKILLS.md (new)
  - docs/library/PROJECTS.md (new)
  - docs/library/INDEX.md (update)
  - ops/lib/manifests/CONTEXT.md (update)
  - TRUTH.md (update - reference new surfaces)
  - SoP.md (update - governance log)

Non-Goals:

- No refactoring of existing project payloads (content within projects/ subdirectories).
- No modification to ops/bin/open or dump logic (only the manifest they read).
- No introduction of PHP code.

9.2 Implementation Steps

1. **Draft SKILL.md:** Create the file at the repo root. Populate headers matching TASK.md. Define the "Skill Library" and "Skill Log" sections.
2. **Draft PROJECT.md:** Create the file at the repo root. Populate headers. Define "Registry Overview" and "Active Projects" sections.
3. **Implement ops/bin/s-learn:** Write the bash script to append structured data to SKILL.md. Ensure chmod +x.
4. **Create Registry:** Initialize ops/lib/project/REGISTRY.md with a Markdown table structure.
5. **Create stela.md Template:** Create a generic stela.md file (likely in projects/ as a reference or ops/lib/project/ as a template source) that defines the schema.
6. **Populate Libraries:** Write the three new manuals in docs/library/.
7. **Register Libraries:** Add the new manuals to docs/library/INDEX.md.
8. **Update Context:** Add SKILL.md and PROJECT.md to ops/lib/manifests/CONTEXT.md.
9. **Update Constitution:** Update TRUTH.md to list the new "Canon surfaces."
10. **Log Governance:** Update SoP.md with the creation of these new root surfaces.

9.3 Verification Plan

- **Context Lint:** Run tools/context_lint.sh. It must pass, confirming that the new files listed in the updated Manifest actually exist.

- **Library Lint:** Run tools/lint_library.sh. It must pass, confirming that docs/library/INDEX.md matches the filesystem.
- **Truth Lint:** Run tools/lint_truth.sh. It must pass, ensuring no forbidden spellings or structure violations in the new files.
- **Functional Test:** Execute ops/bin/s-learn --help to verify the tool is runnable and outputs help text.

10. Implementation Detail: The "No PHP" Constraint

The "No PHP references" requirement is strategic. The Stela platform (PHP-Nuke CE) manages a legacy PHP runtime, but the *governance layer* (Stela itself) is built on Markdown, Bash, and Git. DP-OPS-0010 reinforces this separation.

- **stela.md:** Uses generic types like "module" or "theme" rather than "php-module."
- **SKILL.md:** Skills are defined in natural language or CLI commands, not PHP snippets.
- **S-LEARN-01:** Written entirely in Bash.

This ensures that if the underlying codebase were swapped (e.g., to Python or Node), the governance architecture (TASK, SKILL, PROJECT) would remain valid without modification.

11. Conclusion

DP-OPS-0010 represents a critical maturation of the Stela platform. By evolving from a single-threaded "Task Runner" into a tri-surface "Operating System," the repository gains the memory and foresight required for high-level autonomous operations. The introduction of SKILL.md solves the problem of "Contractor Amnesia," while PROJECT.md solves the problem of "Integrator Myopia."

The rigid adherence to the "Context is Truth" doctrine—updating the manifest only because these new surfaces are indeed truth—ensures that the system remains coherent. Furthermore, the decentralized stela.md pattern lays the groundwork for scaling the project registry infinitely without creating a bottleneck. This architectural specification provides the complete blueprint for executing this transformation while strictly respecting the "No PHP" and "Anti-Drift" constraints.

Component	Role	Governance Surface	Tooling
Execution	Short-term Work	TASK.md	ops/bin/open
Capability	Learned Patterns	SKILL.md	ops/bin/s-learn

Initiative	Long-term Roadmap	PROJECT.md	ops/lib/project/REGISTRY.md

End of Architectural Report.

DP-OPS-0010: Drafted Dispatch Packet

(The following is the drafted DP content, ready for ingestion into TASK.md as per the user query.)

DP-OPS-0010: Establish Root Living Surfaces

0. FRESHNESS GATE (MUST PASS BEFORE WORK)

Base Branch: main

Required Work Branch: work/living-surfaces-init

Base HEAD: [current-hash]

Required local re-check (worker runs; paste results in RESULTS):

- git rev-parse --abbrev-ref HEAD
- git rev-parse HEAD
- git status --porcelain

Worker must already be on the **Required Work Branch** listed above (no branch creation or switching).

STOP if any mismatch.

STOP if the Required Work Branch is missing.

STOP if told to create or switch branches.

I) REQUIRED CONTEXT LOAD (READ BEFORE DOING

ANYTHING)

Worker must confirm loaded before acting:

- OPEN, TRUTH, AGENTS, SoP, CONTEXT, MAP

II) SCOPE & SAFETY

- **Objective:** Establish SKILL.md and PROJECT.md as Root Living Surfaces, create the S-LEARN-01 tool, and standardize project definition via stela.md.
- **Non-Goals:**
  - No edits to existing project payloads.
  - No changes to ops/bin/open or dump logic (surfaces only).
  - No PHP code generation or referencing.

Target Files allowlist (hard gate)

- SKILL.md (new)
- PROJECT.md (new)
- ops/bin/s-learn (new)
- ops/lib/project/REGISTRY.md (new)
- projects/stela.md (new - template)
- docs/library/TASKS.md (new)
- docs/library/SKILLS.md (new)
- docs/library/PROJECTS.md (new)
- docs/library/INDEX.md
- ops/lib/manifests/CONTEXT.md
- TRUTH.md
- SoP.md

Allowlist rule: exact paths only; use (new) prefix only when the DP explicitly allows new files.

- **Conditional Requirements:** If canon/governance surfaces (e.g., TASK.md) are modified, SoP.md MUST be included in the allowlist and updated.
- **Stop Condition (scope):** If any required change falls outside the allowlist, **STOP** and report.
- **Worker Git Authority (non-negotiable):**
  - Workers do not create/switch branches.
  - Workers do not stage/commit/merge/push.
  - Workers only return diffs/results/receipt.

III. EXECUTION PLAN (A–E CANON)

A) STATE

- **Context:** The system currently relies solely on TASK.md for living state. Capabilities (Skills) and Workstreams (Projects) are untracked "dark matter" in the repository.
- **Drift:** Agents may hallucinate skills or project statuses because no SSOT exists for them. projects/ lacks a governance layer.
- **Desired State:** Two new root surfaces (SKILL.md, PROJECT.md) exist, modeled on TASK.md. A tool (s-learn) assists in maintaining Skills. Projects have a local stela.md definition.

B) REQUEST

1. **Create SKILL.md (Root):**
    - Format: Must follow TASK.md header/structure.
    - Sections: Context Load, Active Development, Skill Library, Skill Log.
    - Initial Entry: Add S-LEARN-01 (Self-Learning Tool) to the library.
2. **Create PROJECT.md (Root):**
    - Format: Must follow TASK.md header/structure.
    - Sections: Registry Overview, Active Projects, Project Log.
3. **Create Tool S-LEARN-01 (ops/bin/s-learn):**
    - executable bash script.
    - functionality: appends new skill entries to SKILL.md.
    - logic: simple text appending with timestamp.
4. **Create Registry ops/lib/project/REGISTRY.md:**
    - Static markdown table tracking project IDs, paths, and owners.
5. **Create stela.md Template:**
    - Create projects/stela.md as a base template to be copied.
    - Schema: ID, Type, Dependencies, Context (no PHP refs).
6. **Create Libraries (docs/library/):**
    - TASKS.md: Extract guidance from TASK.md (mechanics).
    - SKILLS.md: Documentation for the new Skill surface.
    - PROJECTS.md: Documentation for the new Project surface.
    - Update docs/library/INDEX.md to register these three new files.
7. **Update Canon:**
    - ops/lib/manifests/CONTEXT.md: Add SKILL.md and PROJECT.md to the Core Set.
    - TRUTH.md: Update Section 1 (Canon surfaces) to include SKILL and PROJECT.

C) CHANGELOG

- Established SKILL.md and PROJECT.md as Root Living Surfaces.
- Created ops/bin/s-learn (S-LEARN-01).
- Established ops/lib/project/REGISTRY.md.

- Created documentation libraries for Tasks, Skills, and Projects.
- Updated CONTEXT.md and TRUTH.md to reflect new architecture.

D) PATCH / DIFF

- (Worker to provide unified diffs for all new and modified files).

E) RECEIPT (REQUIRED)

Verification (MUST RUN)

- ./ops/bin/dump --scope=platform
- bash tools/context_lint.sh (Must pass with new CONTEXT entries).
- bash tools/lint_library.sh (Must pass with new Library entries).
- bash tools/lint_truth.sh (Must pass on new root files).
- bash ops/bin/s-learn --help (Verify tool executability).

Artifact bundle (required directory)

- storage/handoff/

Proof bundle checklist

- storage/handoff/DP-OPS-0010-RESULTS.md
- git diff --stat
- NEXT: Await operator merge.

---

3) CLOSEOUT (MANDATORY)

Worker closeout duties

- [ ] Produce/Update: storage/handoff/DP-OPS-0010-RESULTS.md
- [ ] Update SoP.md (Governance change: Added Root Surfaces).
- [ ] Ensure proof bundle checklist is satisfied.
- [ ] Append a **TASK Work Log** entry.

Works cited

1. dump-full-main-e29ab6b23.txt